
45 Senior Java Backend Interview Questions

With Answers — For 8+ Years Experience Level

Covers JVM internals, collections, multithreading & concurrency, Spring Boot, Spring Security, transactions & AOP, JPA/Hibernate performance, Kafka, microservices & system design, SQL/API performance, and production debugging.

Compiled by Syed Minhaj (Sam) | Senior Java Full Stack Developer / Tech Lead
Toronto, Ontario, Canada

These are not trivia questions. Each one reflects a real design decision, trade-off, or production failure mode you're expected to reason through at senior level — not just define.

Contents

1. Java Core & JVM Internals (6 questions)
2. Collections Internals (5 questions)
3. Multithreading & Concurrency (7 questions)
4. Spring Boot & Spring Security (6 questions)
5. Spring Transactions & AOP (4 questions)
6. JPA / Hibernate Performance (5 questions)
7. Kafka & Event-Driven Architecture (4 questions)
8. Microservices & System Design (5 questions)
9. SQL & API Performance (4 questions)
10. Production Debugging & Engineering Judgment (3 questions)

1. Java Core & JVM Internals

Q1.

Walk through what happens from `java HelloWorld` to the JVM executing bytecode.

Classloader (Bootstrap → Platform → Application) loads the .class file, verifies bytecode structure, prepares static fields, then resolves symbolic references. The interpreter starts executing bytecode; the JIT compiler profiles hot methods (invoked frequently or in tight loops) and compiles them to native machine code via C1 (fast, less optimized) then C2 (slower, aggressively optimized) compilers. This is why a JVM app is slower on startup and speeds up over time — a real consideration when tuning cold-start latency for containerized/serverless Java workloads.

Q2.

Explain the difference between Metaspace and PermGen, and why the change matters operationally.

PermGen (pre-Java 8) stored class metadata in a fixed-size region of the heap, causing frequent `OutOfMemoryError: PermGen space` in apps with heavy class loading (app servers, OSGi, frameworks generating proxies at runtime). Metaspace moved this to native memory, growing dynamically by default. Operationally: you no longer size PermGen manually, but you can still leak native memory if you don't cap Metaspace (-XX:MaxMetaspaceSize) in environments doing dynamic class generation — e.g., heavy CGLIB proxy creation in Spring apps with thousands of beans.

Q3.

Compare G1GC, ZGC, and Shenandoah — when would you pick each for a production service?

G1GC is the default since Java 9: region-based, targets predictable pause times, good general-purpose choice for heaps up to ~32GB. ZGC and Shenandoah are low-latency collectors targeting sub-millisecond pauses even on multi-terabyte heaps, using concurrent evacuation. Pick ZGC/Shenandoah for latency-sensitive services (trading systems, real-time bidding) where p99 pause time matters more than throughput. Pick G1GC for typical REST APIs and batch services where a few hundred ms of occasional pause is acceptable and you want mature, well-understood tuning knobs.

Q4.

What's the difference between a memory leak and excessive memory retention in Java, and how do you diagnose each in production?

A true leak is unreachable-but-referenced memory the GC can never reclaim — e.g., a static `Map` used as a cache with no eviction policy, or listeners registered but never deregistered. Excessive retention is reachable memory that's legitimately large but not actually needed anymore — e.g., a `List` collected in a loop and held in scope longer than necessary. Diagnosis: heap dump via `jmap` or automatic on OOM (-XX:+HeapDumpOnOutOfMemoryError), analyze with Eclipse MAT looking at dominator tree and GC roots. If memory climbs steadily under constant load and never plateaus after full GC, that's a leak signature.

Q5.

Why does `String.intern()` matter for memory, and when would you actually use it in production code?

String literals are interned automatically into the string pool, but strings built at runtime (concatenation, `new String()`, deserialization) are separate heap objects even if equal in content. Calling `.intern()` forces JVM-level deduplication. Real use case: parsing large files or DB result sets with highly repetitive string values (e.g., country codes, status enums stored as strings) — interning can meaningfully cut heap usage. Misuse case: interning high-cardinality or unbounded strings (like UUIDs) can bloat the string pool since pre-Java 7 it lived in PermGen; post-Java 7 it's on the heap and GC'd, but unnecessary interning still adds CPU overhead for no benefit.

Q6.

What's the actual difference between checked and unchecked exceptions from an API design standpoint, not just syntax?

Checked exceptions are a compiler-enforced contract: the caller *must* handle or declare them, which makes sense for recoverable conditions the caller can reasonably act on (e.g., `IOException` — retry, fallback). Unchecked exceptions signal programming errors or conditions the immediate caller can't meaningfully recover from (e.g., `IllegalStateException`). The senior-level debate: checked exceptions don't compose well with functional interfaces and streams, which is why most modern APIs (Spring, reactive libraries) favor unchecked exceptions and push recovery logic to a higher layer via `@ControllerAdvice` or centralized error handling rather than forcing try/catch at every call site.

2. Collections Internals

Q7.

Explain HashMap's internal resizing, and why it can cause an infinite loop or data loss under concurrent access.

HashMap resizes when size exceeds `capacity * loadFactor` (default 0.75), doubling capacity and rehashing entries into new buckets. In Java 7, rehashing used a linked-list insertion-at-head that could form a cycle if two threads resized concurrently, causing an infinite loop on `get()`. Java 8 changed insertion to tail-based and added treeification (buckets with 8+ collisions convert to a red-black tree, $O(\log n)$ instead of $O(n)$), which reduces but does not eliminate concurrency risk — HashMap is still not thread-safe and concurrent structural modification can silently lose entries. Use `ConcurrentHashMap` for concurrent writes, not `Collections.synchronizedMap()` unless you need external locking for compound operations.

Q8.

How does ConcurrentHashMap achieve thread safety without locking the entire map, and what changed between Java 7 and Java 8?

Java 7 used segment-based locking (default 16 segments), so writes to different segments could proceed in parallel but writes to the same segment blocked each other. Java 8 removed segments entirely and locks at the bucket (bin) level using synchronized blocks on the first node, combined with CAS operations for initial insertion — meaning contention only happens when two threads hash to the exact same bucket. This is why Java 8+ ConcurrentHashMap scales far better under high write concurrency than the segmented version.

Q9.

Why does `ArrayList` throw `ConcurrentModificationException`, and why doesn't it always throw it even under concurrent modification?

`ArrayList`'s iterator tracks a `modCount` field incremented on every structural modification. Each `next()` call checks the iterator's expected `modCount` against the list's actual `modCount` and throws if they diverge — this is fail-fast, best-effort detection, not a guarantee. It can miss detection (or throw spuriously) under true concurrent modification because the check itself isn't atomic with the modification. Never rely on CME as a concurrency control mechanism — it's a debugging aid for single-threaded misuse (e.g., removing from a list while iterating with a for-each loop) not a thread-safety guarantee.

Q10.

When would you choose `CopyOnWriteArrayList` over `synchronizedList`, and what's the production trade-off?

`CopyOnWriteArrayList` copies the entire backing array on every write (add/remove/set), giving lock-free, fast reads and safe iteration without external synchronization or CME risk — because iterators work on a snapshot. It's ideal for read-heavy, write-rare scenarios like listener/observer lists or configuration caches refreshed infrequently. It's a poor choice for write-heavy collections since every write is $O(n)$ due to the array copy — I've seen this misused as a general-purpose list in a hot path, causing GC pressure from constant array allocation under load.

Q11.

What's the real difference between `TreeMap` and `LinkedHashMap` when order matters, and what's the complexity trade-off?

`LinkedHashMap` maintains insertion order (or access order if configured) via an internal doubly linked list layered on top of hash bucket storage — $O(1)$ average get/put like `HashMap`, with the ordering as a bonus. `TreeMap` maintains sorted order based on natural ordering or a `Comparator`, backed by a red-black tree — $O(\log n)$ for get/put, always. Use `LinkedHashMap` when you need predictable iteration order (or LRU cache eviction via `removeEldestEntry()`) at `HashMap`-level performance. Use `TreeMap` only when you actually need sorted traversal or range queries (`headMap`, `tailMap`, `subMap`) — reaching for it just for 'ordering' when insertion order would do is a common junior-to-mid-level mistake that costs performance.

3. Multithreading & Concurrency

Q12.

What's the actual difference between `CompletableFuture.supplyAsync()` and a blocking call on a virtual thread — and when does it matter?

`CompletableFuture.supplyAsync()` runs on a platform thread pool (`ForkJoinPool.commonPool` by default) and is designed to avoid blocking those threads, composing async work via callbacks. Virtual threads (Java 21+) let you write plain blocking, sequential code, but the JVM parks the virtual thread and frees the underlying platform (carrier) thread during blocking I/O, so you get similar throughput without callback composition. The difference matters at scale: platform thread pools are capped (typically hundreds), so thread-per-request blocking I/O exhausts the pool under load; virtual threads can scale to hundreds of thousands because they're cheap and don't pin the OS thread during I/O. `CompletableFuture` still matters for CPU-bound parallel composition (fan-out/fan-in) where virtual threads offer no benefit since CPU work still needs a real core.

Q13.

Explain the difference between `volatile` and `synchronized` — what does `volatile` NOT guarantee that people assume it does?

`volatile` guarantees visibility (writes are immediately visible to other threads, no CPU-cache-only writes) and prevents instruction reordering around that variable via a memory barrier. It does NOT guarantee atomicity of compound operations — `counter++` on a volatile int is still a read-modify-write that can race. `synchronized` guarantees both visibility and atomicity for the guarded block, at the cost of mutual exclusion. Classic production bug: using `volatile boolean flag` for a shutdown signal is correct; using `volatile int counter` for a request counter under concurrent increments is not — you need `AtomicInteger` or synchronization.

Q14.

Why does `ThreadLocal` cause memory leaks specifically in thread-pooled environments like Tomcat or an `ExecutorService`, and not in short-lived threads?

`ThreadLocal` stores values in a map owned by the Thread object itself (`ThreadLocalMap`), keyed by weak references to the `ThreadLocal` instance but with strong references to the stored value. In a thread pool, threads are reused indefinitely, so if you don't call `.remove()` after use, the stored value stays referenced by the thread for its entire lifetime — accumulating across every request that reuses that thread. In short-lived threads, the thread (and its `ThreadLocalMap`) dies and gets GC'd naturally, so the leak never accumulates. Real production case: storing a request-scoped user context in `ThreadLocal` for MDC logging without clearing it in a `finally` block leaked user data across unrelated requests handled by the same pooled thread — a correctness bug and a data-leak security issue, not just a memory one.

Q15.

Explain the producer-consumer deadlock risk with a bounded `BlockingQueue`, and how you'd design around it.

A bounded queue blocks producers when full and consumers when empty. Deadlock risk arises when a consumer thread also needs to produce to complete its work (e.g., a worker that processes an item and then submits a follow-up task to the same queue) — if the queue is full, the consumer blocks trying to add, but nothing is consuming because the only consumer is blocked. Fix: separate queues per stage of the pipeline, use `offer()` with timeout instead of blocking `put()`, or size the thread pool so producer and consumer roles are never the same thread for the same queue.

Q16.

What's the difference between `Executors.newFixedThreadPool()` and manually configuring `ThreadPoolExecutor`, and why do most style guides now discourage the former?

`Executors.newFixedThreadPool(n)` uses an unbounded `LinkedBlockingQueue` internally — under sustained load exceeding thread capacity, tasks queue indefinitely instead of applying backpressure, which can exhaust memory before you ever see thread starvation symptoms. Manually configuring `ThreadPoolExecutor` lets you set a bounded queue with an explicit `RejectedExecutionHandler` (e.g., `CallerRunsPolicy` to apply backpressure to the caller), giving you a controlled failure mode instead of an OOM. This is why production Spring Boot services typically define `ThreadPoolTaskExecutor` beans explicitly rather than relying on default factory methods.

Q17.

How would you diagnose a thread pool exhaustion issue in a live production service without restarting it?

Take a thread dump (`jstack`` or `kill -3`` on the JVM process) and look for threads in `WAITING`/TIMED_WAITING`` state stuck on the same lock or queue — a large cluster of threads blocked on one resource (a slow downstream HTTP call, a DB connection pool, a synchronized block) indicates the bottleneck. Cross-reference with the connection pool's active/idle metrics (HikariCP exposes these via Micrometer) — if thread count is maxed and DB connections are also maxed with long checkout times, the DB is the actual root cause, not the thread pool config itself. Increasing pool size in that case just moves the bottleneck downstream and can worsen DB contention.

Q18.

What's a happens-before relationship, and why does it matter more than 'is it synchronized' when reasoning about correctness?

Happens-before is the Java Memory Model's formal guarantee about ordering and visibility between actions across threads — e.g., a write before releasing a lock happens-before a read after acquiring that same lock; a write to a volatile field happens-before a subsequent read of it. The reason it matters more than 'is it synchronized': two threads can both use synchronization correctly but on different locks and still have no happens-before relationship between their operations, meaning visibility isn't guaranteed. Reasoning in terms of happens-before edges (not just 'did I use synchronized somewhere') is what separates engineers who can debug subtle race conditions from those who add synchronized until symptoms disappear.

4. Spring Boot & Spring Security

Q19.

How does Spring Boot auto-configuration decide which beans to create, and how would you debug a bean that isn't being created when you expect it?

Auto-configuration classes are loaded via `spring.factories`` (or `AutoConfiguration.imports`` in Boot 2.7+) and each is gated by `@Conditional`` annotations — `@ConditionalOnClass`` (is the dependency on the classpath), `@ConditionalOnMissingBean`` (don't override a user-defined bean), `@ConditionalOnProperty``, etc. To debug a missing bean: run with `--debug`` or set `logging.level.org.springframework.boot.autoconfigure=DEBUG`` to get the auto-configuration report, which lists every conditional evaluation and exactly why each auto-config class was or wasn't applied — this is far faster than guessing from stack traces.

Q20.

Explain the actual request flow through Spring Security's filter chain for a JWT-based stateless API.

The request hits `DelegatingFilterProxy` which routes into the `FilterChainProxy` holding the ordered `SecurityFilterChain`. For JWT auth, a custom filter (extending `OncePerRequestFilter`, placed before `UsernamePasswordAuthenticationFilter`) extracts the token, validates signature and expiry, and if valid, builds an `Authentication` object and sets it on the `SecurityContextHolder` for that request thread. Downstream, `@PreAuthorize` and method security intercept via AOP proxies checking the `SecurityContext`. Because it's stateless, `SecurityContextHolder` uses `MODE_THREADLOCAL` by default and must be cleared per request — this is also why propagating security context into async threads (e.g., inside `@Async` methods or a custom `ExecutorService`) requires explicitly copying it; it does not propagate automatically.

Q21.

What's the actual security risk of storing JWTs in localStorage vs. an httpOnly cookie, and how does that change your CSRF strategy?

localStorage is accessible to any JavaScript running on the page, so an XSS vulnerability anywhere in the frontend can exfiltrate the token directly. httpOnly cookies are inaccessible to JavaScript, mitigating XSS token theft, but cookies are automatically attached to requests by the browser, reintroducing CSRF risk that a header-based Authorization token doesn't have. If you use httpOnly cookies for JWT storage, you need CSRF protection (SameSite=Strict/Lax plus a CSRF token for state-changing requests); if you use localStorage with an Authorization header, you're CSRF-safe by construction but XSS-exposed. This is a real architectural trade-off, not a solved problem — the 'correct' answer depends on your actual threat model.

Q22.

How does `@PreAuthorize` actually enforce method-level security, and why can it silently fail to trigger?

`@PreAuthorize` relies on Spring AOP proxies (same mechanism as `@Transactional`), evaluated before method invocation via a `MethodSecurityInterceptor`. It silently fails to trigger for the same reason `@Transactional` does: internal self-invocation bypasses the proxy entirely, so calling an `@PreAuthorize`-annotated method from another method in the same class skips the security check with no error or warning. It also requires `@EnableMethodSecurity` (or the older `@EnableGlobalMethodSecurity`) to be active, and won't apply to private methods or static methods, since CGLIB/JDK dynamic proxies can't intercept those.

Q23.

What's the difference between authentication and authorization failures in Spring Security's exception model, and how should your API surface them?

`AuthenticationException` (and subtypes like `BadCredentialsException`) represents 'who are you' failures, handled by an `AuthenticationEntryPoint`, typically resulting in a 401. `AccessDeniedException` represents 'I know who you are, but you can't do this' failures, handled by an `AccessDeniedHandler`, typically resulting in a 403. A common production mistake is returning 403 for both cases — this leaks information (confirming a resource exists but is forbidden vs. genuinely requiring login) and makes client-side error handling ambiguous. Explicitly wiring both handlers in your `SecurityFilterChain` keeps the contract clean for API consumers.

Q24.

How would you design rate limiting for a public API in Spring Boot, and what are the trade-offs between in-memory and distributed approaches?

In-memory (e.g., Bucket4j with a local cache) is simple and fast but breaks down across multiple instances behind a load balancer — each instance enforces its own limit independently, so effective rate limit is (configured limit × instance count). Distributed rate limiting (Redis-backed token bucket or sliding window, via Bucket4j's Redis integration or a dedicated gateway like Kong/Spring Cloud Gateway) enforces a true global limit but adds a network hop and a dependency that becomes a single point of failure if not made resilient. For most enterprise APIs behind a gateway, pushing rate limiting to the API gateway layer rather than application code is the more maintainable architecture — it decouples a cross-cutting concern from business logic.

5. Spring Transactions & AOP

Q25.

Why does `@Transactional` silently not roll back when an internal method call throws an exception, and what's the actual fix?

`@Transactional` is implemented via a Spring AOP proxy wrapping the bean. Calling an annotated method from another method in the *same class* invokes `this.method()` directly, bypassing the proxy entirely — so no transaction is ever opened for that call, and there's nothing to roll back. The clean fix is structural: move the transactional method into a separate collaborator bean and inject it, so the call goes through the proxy. A less clean but sometimes pragmatic fix is self-injection of the proxy (`@Lazy` injected reference to itself) — acceptable for legacy code you can't easily restructure, but worth flagging as technical debt in review.

Q26.

What's the difference between `REQUIRED` and `REQUIRES_NEW` propagation, and give a real scenario where using the wrong one causes a production bug.

`REQUIRED` (default) joins an existing transaction if one is active, or creates a new one if not — a rollback anywhere in that shared transaction rolls back everything. `REQUIRES_NEW` always suspends any existing transaction and starts a fresh, independent one, committing or rolling back separately. Real bug: an audit-logging method using `REQUIRED` inside a larger business transaction — if the business logic later throws and rolls back, the audit log entry rolls back too, even though you wanted an immutable record that the attempt happened regardless of outcome. That method needed `REQUIRES_NEW` so the audit write commits independently of the outer transaction's fate.

Q27.

Explain the difference between `readOnly = true` on `@Transactional` and simply not modifying data — does it actually do anything?

`readOnly = true` is a hint, not an enforced constraint at the JDBC level in most databases — it doesn't prevent writes at the Java layer. What it actually does with Hibernate: it configures the `FlushMode` to avoid unnecessary dirty-checking and flush cycles, and can be used by the connection pool or a read-replica routing `DataSource` to direct traffic to a read replica instead of the primary. In a system with read replicas, forgetting `readOnly = true` on genuinely read-only service methods means every query hits the primary unnecessarily — a real, measurable scalability cost in high-read systems, not just a style preference.

Q28.

What isolation level do most production Spring Boot apps run at by default, and what specific bug class does it leave you exposed to?

Most default to the database's default isolation level — for PostgreSQL and MySQL/InnoDB, that's READ COMMITTED. It prevents dirty reads but not non-repeatable reads or phantom reads: reading the same row twice within one transaction can return different values if another transaction commits in between. Real-world impact: a 'check-then-act' pattern (check inventory count, then decrement it) under READ COMMITTED is a race condition unless you use `SELECT ... FOR UPDATE` (pessimistic locking) or optimistic locking via a `@Version` field — relying on the default isolation level alone to prevent double-booking or overselling is a subtle production bug that only surfaces under real concurrent load, not in testing.

6. JPA / Hibernate Performance

Q29.

Explain the N+1 query problem with a concrete example, and the actual trade-offs between the three main fixes.

Fetching a list of `Order` entities, then accessing `order.getCustomer().getName()` in a loop triggers one query for the orders and one additional query per order to lazily fetch its customer — N+1 total queries. Fix 1: `JOIN FETCH` in JPQL — single query, but can cause cartesian-product row duplication when fetching multiple collections at once. Fix 2: `@EntityGraph` — declarative, cleaner for reuse across repository methods, same underlying join behavior. Fix 3: batch fetching (`@BatchSize` or `hibernate.default\_batch\_fetch\_size`) — issues one query per batch of N parent IDs instead of one per row, better when you can't safely use a single join (e.g., multiple lazy collections). Picking JOIN FETCH by default for every association without considering row multiplication is a common cause of duplicate-row bugs in production result sets.

Q30.

What's the difference between first-level and second-level cache in Hibernate, and why does misunderstanding this cause stale-data bugs?

First-level cache is the `Session`/`EntityManager`-scoped cache — always on, guarantees you get the same object instance within one transaction, and is cleared when the session closes. Second-level cache (EHCache, Caffeine, Redis via a provider) is `SessionFactory`-scoped, shared across sessions and potentially across JVM instances if using a distributed cache provider — it persists across transactions. The stale-data bug: teams enable second-level cache for an entity to reduce DB load, but forget that direct SQL/JDBC writes (batch jobs, other services writing to the same table) bypass Hibernate entirely and don't invalidate the cache, leaving the application serving stale data indefinitely until TTL expiry or a manual cache evict.

Q31.

Explain optimistic locking with `@Version` — what actually happens on a conflict, and how should the application handle it?

Hibernate adds a version column checked and incremented on every UPDATE; the generated SQL includes `WHERE id = ? AND version = ?`. If another transaction updated the row first, zero rows match, Hibernate detects the mismatch and throws `OptimisticLockException`. This is not caught and silently retried by default — the application must catch it explicitly and decide the recovery strategy: retry the entire business operation with fresh data (correct for most cases), or surface a 409 Conflict to the client for user-driven merge (correct for collaborative editing scenarios). Letting `OptimisticLockException` propagate as an unhandled 500 error is a common oversight that turns a normal concurrency event into a false alarm in production monitoring.

Q32.

Why can `SELECT ... FOR UPDATE` deadlock two services that both individually use `@Transactional` correctly?

Pessimistic row locks acquired via `FOR UPDATE` are held until transaction commit/rollback. If Service A locks row 1 then tries to lock row 2, while Service B (or a concurrent request in the same service) locks row 2 then tries to lock row 1, both transactions wait on each other indefinitely — a classic deadlock, and it's correct `@Transactional` usage on both sides that doesn't prevent it, because the annotation manages transaction boundaries, not lock acquisition order. The database will typically detect this and abort one transaction with a deadlock exception, but the fix is architectural: always acquire locks on multiple rows in a consistent, defined order (e.g., always lock by ascending primary key) across every code path that touches those rows.

Q33.

What's the practical difference between `EAGER` and `LAZY` fetching, and why is defaulting everything to `EAGER` a common performance mistake?

`EAGER` loads the association immediately with the parent entity (via a join or a separate immediate query); `LAZY` defers loading until the association is actually accessed, throwing `LazyInitializationException` if accessed outside an active session/transaction. Defaulting everything to `EAGER` 'to avoid LazyInitializationException errors' — a common junior/mid-level habit — means every simple entity fetch pulls in its entire object graph even when unused, causing bloated queries and unnecessary joins across every code path, not just the ones that need the association. The correct pattern is `LAZY` by default (JPA's actual default for `@OneToMany`/`@ManyToMany`) with deliberate `JOIN FETCH` or `@EntityGraph` on the specific queries that need the association eagerly.

7. Kafka & Event-Driven Architecture

Q34.

Explain exactly-once semantics in Kafka — what does it actually guarantee, and where does it commonly break in real systems?

Kafka's exactly-once semantics (via idempotent producers and transactional writes) guarantees exactly-once delivery **within Kafka** — from producer to topic to consumer offset commit, atomically. It does NOT guarantee exactly-once processing of side effects outside Kafka. If your consumer processes a message and calls an external REST API or writes to a database as a side effect, and then crashes before committing the Kafka offset, the message is redelivered and that external side effect runs twice. The real fix is designing consumers to be idempotent at the business level — e.g., using a unique event ID with a database uniqueness constraint or upsert logic — rather than relying on Kafka's internal guarantee to cover your entire processing pipeline.

Q35.

Why does consumer group rebalancing cause duplicate processing, and how do you minimize its impact in production?

When a consumer joins or leaves a group (deploy, crash, scale event), partitions are reassigned. With the default eager rebalancing protocol, ALL consumers stop processing during the rebalance ('stop-the-world'), and messages processed but not yet offset-committed before the rebalance get reprocessed by whichever consumer picks up that partition next. Minimizing impact: use `CooperativeStickyAssignor`` (incremental rebalancing, only reassigns the specific partitions that need to move, not all of them), tune ``max.poll.interval.ms`` and ``session.timeout.ms`` to avoid false-positive consumer-dead detection during legitimate long processing, and design idempotent consumers so duplicate delivery during rebalance doesn't corrupt state.

Q36.

What's the real difference between choosing Kafka vs. RabbitMQ for a given use case, beyond 'Kafka is for streaming'?

RabbitMQ is a traditional message broker: messages are removed once consumed/acked, supports complex routing (topic/fanout/direct exchanges), and is well-suited for task queues and RPC-style patterns where you want per-message routing logic and lower operational overhead. Kafka is a distributed append-only log: messages persist for a configured retention period regardless of consumption, allowing multiple independent consumer groups to replay the same data at different times, which is essential for event sourcing, audit trails, or feeding both a real-time service and a batch analytics pipeline from the same event stream. The deciding question isn't throughput — it's whether you need replay/multiple-consumer-group semantics (Kafka) or flexible per-message routing with simpler ops (RabbitMQ).

Q37.

How do you handle a 'poison pill' message that repeatedly crashes your Kafka consumer, without stopping the pipeline entirely?

Without handling, a message that always throws (malformed payload, unhandled edge case) causes infinite retry-and-crash, blocking that partition's offset from ever advancing and stalling every message behind it. Production pattern: wrap processing in error handling that, after N retries (using Spring Kafka's ``DefaultErrorHandler`` with a ``BackOff`` policy), routes the failing message to a dead-letter topic via ``DeadLetterPublishingRecoverer`` and commits the offset to unblock the partition. The dead-letter topic is then monitored and reprocessed manually or via a separate remediation consumer — critical because silently dropping the message loses data, but blocking forever takes down the whole pipeline for one bad record.

8. Microservices & System Design

Q38.

How do you handle distributed transactions across microservices when two-phase commit isn't viable?

The Saga pattern: break the transaction into a sequence of local transactions, each with a corresponding compensating action to undo it if a later step fails. Choreography-based sagas (each service publishes events, others react) work well for simple, few-step flows but get hard to reason about as steps grow — the overall flow lives implicitly across many services. Orchestration-based sagas (a central coordinator explicitly calls each step and its compensations) trade a single point of coordination for much clearer flow visibility and easier debugging, which is usually worth it once you have more than 3-4 steps or need to track saga state for observability.

Q39.

Explain the circuit breaker pattern — what specific production failure mode does it prevent that a simple timeout doesn't?

A timeout limits how long you wait for one failing call, but if a downstream service is degraded, every new request still attempts the call, times out, and ties up a thread/connection while waiting — under load this cascades into thread pool exhaustion in the calling service, turning one dependency's failure into your own outage. A circuit breaker (Resilience4j) tracks failure rate and, once a threshold is exceeded, 'opens' and fails fast without attempting the call at all for a cooldown period, protecting your own resources and giving the downstream service room to recover instead of being hammered by retries during its outage.

Q40.

What's the trade-off between synchronous REST calls and asynchronous event-driven communication between microservices, beyond 'loose coupling'?

Synchronous REST gives you immediate consistency and simple request-response reasoning, but couples availability — if service B is down, every synchronous caller of B fails immediately, and latency compounds across chained calls (A calls B calls C). Async event-driven communication decouples availability (a queue buffers during downstream outages) and allows independent scaling, but introduces eventual consistency you must design for explicitly — the caller can't know 'did this succeed' synchronously, which pushes complexity into idempotency, correlation IDs for tracing, and UI/UX patterns that tolerate delayed confirmation. Neither is universally correct; the decision should follow whether the business operation genuinely needs an immediate answer.

Q41.

How would you design idempotency for a payment API that might receive the same request twice due to client retries?

Require the client to send a unique idempotency key per logical operation (not per HTTP request) in a header. On the server, before processing, atomically check-and-insert the key into a store (DB unique constraint or Redis `SETNX`) — if the key already exists, return the cached response from the first successful execution instead of reprocessing. Critical detail: the check-and-insert plus the actual business operation need to be handled correctly for the in-flight case (a second request arriving while the first is still processing) — typically by returning a 409/425 and asking the client to retry after a short delay, rather than allowing both to proceed.

Q42.

What's the actual difference between a load balancer doing health checks and a service mesh's circuit breaking, and when do you need both?

A load balancer's health check is binary and coarse — is this instance up or down, remove it from rotation if it fails checks. It doesn't understand per-request failure patterns, latency degradation short of full failure, or protect the caller from cascading timeouts. A service mesh (Istio, Linkerd) operating at the sidecar/proxy level provides fine-grained per-service circuit breaking, retries with backoff, and traffic shaping without application code changes. You typically need both: the load balancer handles instance-level up/down routing, while the mesh (or an in-process library like Resilience4j if you don't run a mesh) handles request-level resilience patterns within otherwise-healthy instances.

9. SQL & API Performance

Q43.

A query that used to run in 50ms now takes 3 seconds after the table grew — what's your systematic diagnostic approach?

First, run `EXPLAIN ANALYZE` to see the actual execution plan, not the estimated one — look specifically for a sequential scan where you'd expect an index scan, which usually means either a missing index, an index that exists but isn't being used (function on the indexed column, implicit type casting, or leading wildcard `LIKE '%x%'`), or stale table statistics causing the query planner to make a bad choice. Second, check if the growth crossed a threshold where the planner switched strategies (e.g., the optimizer decided a sequential scan is cheaper than an index scan for a large fraction of rows). Third, run `ANALYZE` to refresh statistics before concluding you need a new index — sometimes stale stats alone cause this regression with no schema change needed.

Q44.

Explain why pagination with `OFFSET` gets progressively slower on large tables, and what the production fix is.

`OFFSET 100000 LIMIT 20` still requires the database to scan and discard the first 100,000 rows before returning the next 20 — cost grows linearly with offset depth, so page 5000 is dramatically slower than page 1 even though both return the same number of rows. The fix is keyset (cursor-based) pagination: instead of `OFFSET`, use `WHERE id > :last_seen_id ORDER BY id LIMIT 20`, which uses an index seek instead of a scan-and-discard, giving constant-time performance regardless of page depth. The trade-off is you lose the ability to jump to an arbitrary page number, which is usually an acceptable UX change for infinite-scroll or 'next page' interfaces but not for a 'jump to page 47' UI.

Q45.

What's the real cost difference between a database connection pool that's too small vs. too large, and how do you find the right number?

Too small: requests queue waiting for a connection, adding latency and eventually timing out under load even if the database itself has capacity to spare. Too large: each connection consumes DB-side memory and can increase context-switching/lock contention on the database, and paradoxically can *reduce* throughput past a certain point because the DB spends more time coordinating than executing queries. HikariCP's own guidance is that pool size should be much smaller than people assume — often derived from `connections = ((core_count * 2) + effective_spindle_count)` as a starting formula, then tuned against actual observed wait time and utilization metrics, not guessed from request volume.

Q46.

How would you design an API to support high read throughput on data that's expensive to compute, without serving stale results forever?

Cache-aside pattern with an explicit TTL is the baseline: compute on cache miss, serve from cache (Redis/Caffeine) on hit. For data where staleness has real business cost but recomputation is expensive, add a background refresh (refresh-ahead) that recomputes before TTL expiry so requests never hit a cold cache during peak load. For data with strict consistency requirements after a write, use explicit cache invalidation on write rather than relying purely on TTL — this requires the write path to know about and invalidate the correct cache keys, which is the actual hard part of cache design, not the caching mechanism itself.

10. Production Debugging & Engineering Judgment

Q47.

A service's p99 latency spiked from 200ms to 4s with no code deploy and no traffic increase — walk through your investigation.

Rule out infrastructure first: check GC logs for a sudden increase in GC pause frequency/duration (a memory leak or a change in allocation pattern from an upstream data shape change can trigger this without a deploy). Check downstream dependency latency — a degraded database, third-party API, or another microservice can silently push your p99 up even though your code didn't change. Check for a 'noisy neighbor' if running on shared infrastructure (Kubernetes node under CPU pressure from another pod). Correlate the exact spike timestamp against deploy logs of **dependencies** (not just your own service), infrastructure change logs, and any scheduled batch job or cron that might be contending for the same resources — 'no code deploy' doesn't rule out 'something in the environment changed'.

Q48.

What's your process for reviewing a pull request that 'passes all tests' but you still have a bad feeling about?

Passing tests confirm the code does what the tests check, not that the tests check the right things. I look specifically for what's **not** tested: concurrent access paths, failure/timeout scenarios for external calls, and boundary conditions on collections and pagination. I check whether new database queries have appropriate indexes for the access pattern, and whether the change introduces a new N+1 pattern or removes an existing safeguard (like a `@Transactional` boundary or an idempotency check) without an equivalent replacement. If the 'bad feeling' persists after that, I ask the author to walk through the failure mode explicitly in the PR description — often the act of writing it out surfaces the gap themselves.

Q49.

How do you decide when a performance problem needs a code fix vs. when it needs an architecture change?

If the fix is local — a missing index, an N+1 query, an inefficient loop — that's a code fix, and I'd expect to see a measurable improvement with low risk and no design discussion needed. If fixing it properly requires changing how data flows between components — e.g., a synchronous call chain that can't scale no matter how optimized each individual call is, because the latency is additive by design — that's an architecture problem, and optimizing the code further just delays hitting the same wall. The signal I look for: if you've already fixed the obvious code-level issues and the bottleneck persists, or if the fix would require the same optimization to be reapplied every time volume grows, you're looking at a structural limit, not a bug.

Practicing these won't just help you pass an interview — it'll make you a better engineer for the systems you're already running in production.

Compiled by Syed Minhaj (Sam) — Senior Java Full Stack Developer / Tech Lead, Toronto, ON. Connect on LinkedIn for more posts on Java internals, Spring, and system design.